

Resilient Order & Payment Platform

Project Management Plan

Java Backend Portfolio Project — Idempotency, Concurrency & Payment Reliability

Target role: Java Backend Developer | Stretch: Full Stack
Capacity: 10–15 hrs/week | Duration: 5–6 weeks

1. Project Overview

This project is a Spring Boot backend that processes e-commerce orders and payments reliably under real-world failure conditions — duplicate requests, concurrent inventory access, and payment failures. The headline feature is idempotent request handling: the system must guarantee that a retried request never creates a duplicate order or double charge.

Unlike a standard CRUD e-commerce clone, this project is scoped around backend engineering depth: transaction boundaries, optimistic locking, state machines, and audit-safe data design. Every feature exists to generate a concrete, defensible answer to an interview question.

1.1 Objectives

- Demonstrate idempotent API design using an idempotency-key pattern.
- Demonstrate concurrency-safe inventory management using optimistic locking.
- Demonstrate secure authentication and authorization using JWT and OAuth2 with RBAC.
- Demonstrate transactional integrity across multi-step business operations.
- Produce a deployable, documented, containerized backend suitable for a resume and live demo link.

1.2 Tech Stack

- Language / Framework: Java, Spring Boot, Spring MVC, Spring Web
- Data: Spring Data JPA, Hibernate, Spring JDBC, MySQL
- Security: Spring Security, JWT, OAuth2
- Testing: JUnit 5, Mockito
- Docs & Delivery: springdoc-openapi (Swagger), Docker, docker-compose
- Version Control: Git, with commit history from project inception

1.3 Out of Scope (for this pass)

The following are deliberately excluded to keep the core deep rather than the surface wide. They are documented here as a future roadmap, not forgotten scope:

- Redis-backed distributed locking / caching
- Kafka or other async messaging
- OAuth2 social login (Google/GitHub) beyond basic OAuth2 resource server setup
- React/Angular frontend

2. Roadmap Summary

Phase	Focus	Timeframe	Key Outcome
0	Setup	Day 1	Project boots, Git initialized, DB connected
1	Auth Foundation	Week 1	JWT login/register, roles enforced
2	Product & Order Core	Week 2	Happy-path order flow works end to end
3	Idempotency (headline)	Week 3	Duplicate requests provably safe
4	Concurrency Safety	Week 4	No overselling under simultaneous load
5	Payment State Machine	Week 5	Auditable payment lifecycle, global error handling
6	Polish & Ship	Week 6	Docs, Docker, optional live deployment

3. Phase-by-Phase Plan

Phase 0: Setup

Day 1 • ~1–2 hrs | Goal: Environment ready, project boots

Task	Est. Hours	Deliverable / Definition of Done
Generate project via Spring Initializr (Web, Data JPA, Security, MySQL Driver, Validation, Lombok)	0.5	Project compiles and runs locally
Initialize Git repository, first commit	0.25	Repo exists with meaningful initial commit
Configure application.yml and connect to local MySQL	0.5	App boots with DB connection confirmed

Phase 1: Auth Foundation

Week 1 • ~8–10 hrs | Goal: Register/login working with JWT, roles enforced

Task	Est. Hours	Deliverable / Definition of Done
Design User entity and Role enum (USER, ADMIN)	1	Entity + migration/schema in place
Build register and login endpoints with BCrypt password hashing	2.5	Can register and log in via Postman
Implement JWT generation, parsing, and validation	3	Valid token issued and accepted
Configure Spring Security filter chain and role-based endpoint protection	2.5	Protected endpoint rejects invalid/missing token
Manual test pass with Postman collection	1	Auth flow demoed end to end

Phase 2: Product & Order Core

Week 2 • ~10–12 hrs | Goal: Happy-path CRUD and order creation working

Task	Est. Hours	Deliverable / Definition of Done
Design Product entity and CRUD endpoints (ADMIN-only writes)	3	Products can be created, listed, updated
Design Order and OrderItem entities and relationships	2	Schema supports multi-item orders
Implement POST /api/orders happy path	3	Order created successfully with valid items

Task	Est. Hours	Deliverable / Definition of Done
(no idempotency yet)		
Implement GET /api/orders/{id} and GET /api/orders (own history)	1.5	User can retrieve their own orders
Write first JUnit + Mockito test for order-total calculation logic	2	Test passes in isolation from DB

Phase 3: Idempotency — Headline Feature

Week 3 • ~8–10 hrs | Goal: Duplicate requests cannot create duplicate orders

Task	Est. Hours	Deliverable / Definition of Done
Design IdempotencyKey table (key, request hash, response body, status)	1.5	Schema supports key lookup and storage
Implement interceptor/filter to check key before controller logic runs	3	Repeated key short-circuits reprocessing
Handle same-key-different-payload case with 409 Conflict	1.5	Conflicting reuse is rejected, not silently processed
Store response against key after successful processing	1	Replayed request returns identical stored response
Write test: fire identical request twice, assert single order created	2	Test passes and documents the guarantee

Phase 4: Concurrency Safety

Week 4 • ~8–10 hrs | Goal: No overselling when two orders race for the same stock

Task	Est. Hours	Deliverable / Definition of Done
Add @Version field to Product for optimistic locking	1	Version column present and incrementing on update
Catch OptimisticLockException and map to a clean out-of-stock response	2	Race loser receives clear, handled error
Define and review @Transactional boundaries across order placement	2	Partial failure rolls back cleanly
Write concurrency test: two simultaneous requests for last unit of stock	3	Only one request succeeds, verified by test

Phase 5: Payment State Machine

Week 5 • ~8–10 hrs | Goal: Payment lifecycle is auditable, not a boolean flag

Task	Est. Hours	Deliverable / Definition of Done
Design Payment entity with status enum (PENDING, PROCESSING, SUCCESS, FAILED)	1.5	Schema supports full lifecycle
Build mock payment processor service (simulated success/failure)	2.5	Payment attempts transition status correctly
Implement global exception handling via @ControllerAdvice	2.5	Custom exceptions map to correct HTTP status codes app-wide
Write tests covering success, failure, and retry paths	2	State transitions verified by tests

Phase 6: Polish & Ship

Week 6 • ~6–8 hrs | Goal: Project is documented, containerized, and demo-ready

Task	Est. Hours	Deliverable / Definition of Done
Add springdoc-openapi and verify Swagger UI	1.5	All endpoints visible and testable via Swagger
Write Dockerfile and docker-compose.yml (app + MySQL)	2.5	docker-compose up runs the full stack
Write README with architecture diagram and design-decision rationale	2.5	README explains idempotency, locking, and transaction choices
Optional: deploy to free-tier host (Render/Railway)	1.5	Live demo link available

4. Core Data Model

High-level entity relationships. Full field-level schema to be finalized during Phase 2–3.

- User — has many Orders; has a Role
- Product — has a stock quantity and a @Version column for optimistic locking
- Order — belongs to a User; has many OrderItems; has one Payment
- OrderItem — belongs to an Order; references a Product
- Payment — belongs to an Order; has a status enum and a transition history
- IdempotencyKey — standalone table keyed by client-supplied key; stores request hash, response, status

5. Core API Surface

Method & Path	Purpose	Notes
POST /api/auth/register	Create a new user account	Password hashed with BCrypt
POST /api/auth/login	Authenticate and issue JWT	Returns access token
GET /api/products	List products	Public, paginated
POST /api/products	Create a product	ADMIN only
POST /api/orders	Place an order	Requires Idempotency-Key header
GET /api/orders/{id}	Retrieve a single order	Owner or ADMIN only
GET /api/orders	List current user's orders	Paginated
POST /api/payments/{orderId}	Trigger/simulate payment processing	Drives Payment state machine
GET /api/payments/{id}	Retrieve payment status/history	Owner or ADMIN only

6. Risks & Mitigations

Risk	Impact	Mitigation
New to testing (JUnit/Mockito) — risk of skipping tests under time pressure	Concurrency and idempotency claims become unverifiable	Write the two ‘proof’ tests (duplicate request, concurrent stock) as non-negotiable deliverables, not stretch goals
New to Docker — risk of it eating a full week	Delays shipping, discourages finishing	Budget fixed 2–3 hrs in Phase 6 only; use a single known-good docker-compose template rather than researching from scratch
Scope creep (adding Redis/Kafka/React mid-build)	Core feature (idempotency) ships unfinished	Explicitly logged as Out of Scope in Section 1.3; revisit only after Phase 6 is complete
10–15 hrs/week is inconsistent across weeks	Timeline slips past 6 weeks	Each phase has a standalone ‘stop point’ that is still demo-able if the next phase is delayed

7. Definition of Done (Project-Level)

- A duplicate request with the same Idempotency-Key never creates a second order — proven by an automated test.
- Two simultaneous purchase requests for the last unit of stock never both succeed — proven by an automated test.
- All endpoints are protected by role-appropriate JWT authorization.
- Payment status is tracked as an auditable history, not overwritten in place.
- The entire stack (app + MySQL) starts with a single docker-compose up command.
- README explains the reasoning behind idempotency, locking strategy, and transaction boundaries — not just how to run the app.

8. Resume Line (Target Output)

Resilient E-Commerce Order & Payment Platform

Developed a Spring Boot backend implementing idempotent order and payment processing, transactional inventory reservation, JWT/OAuth2 authentication, role-based authorization, and concurrency-safe order management using JPA/Hibernate and MySQL.